Advanced Data Structures:
Stacks, Queues, Linked Lists

PAGE NO.....
DATE / /

1. Stacks

→ A stack is Last-In-First-Out (LIFO) data structure. The last element added is the first one to be removed

Operations on a Stack.

- push: Add an element to the top of the stack
- Pop: Remove and return the top element of the stack.
- Peak: Return the top element without removing it
- is Empty: check if the stack is empty.

Implementing a stack in Python:

```python
#push elements
stack.append(1)
stack.append(2)
stack.append(3)
print(stack) #output: [1,2,3]

# Pop elements
print(stack.pop()) # output: 3.
print(stack.pop()) # output: 2.

# Peek
print(stack[-1]) # output: 1
```

```
# check if stack is empty
print(len(stack) == 0) # output: False.
```

2) Queues

→ A queue is a First-In-First-Out (FIFO) data structure. The first element added in the first one to be removed.

Operations on a Queue.

- Enqueue: Add an element to the back of the queue
- Dequeue: Remove and return the front element of the queue.
- Peek: Return the front element without removing it.
- IsEmpty: Check if the queue is empty.

Implementing a Queue in Python

```
from collection import deque
    queue = deque() # create a queue.
# Enqueue elements.
  queue.append(1)
  queue.append(2)
  queue.append(3)
  print(queue) # output: deque([1,2,3])
```

PAGE NO.....
DATE: / /

Dequeue elements
print(queue.popleft()) # output: 1
print(queue.popleft()) # output: 2

Peek
print(queue[0]) # output: 3

check if queue is empty
print(len(queue) == 0) # output: False.

3) Linked lists

Why queue = deque()?

→ deque is a class from the collections module that provides a double-ended queue. It supports:
- fast appends and pops from both the front and back.
- Efficient memory useage.

Why Not use a List for Queue?

→ queue = []
→ you cant use popleft in queue[].

PAGE NO.....
DATE / /

3. Linked Lists

→ A linked list is linear data structures where each element (node) contains data and a references (pointers) to the next node.

→ Types of linked lists

- single linked list
  Each node points to the next node.

- Doubly linked list
  Each node points to both the next and previous nodes.

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None  # Attribute holds an object reference.

    def append(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node
```

this node does not point to other node, means, this will store a reference to the next node.

note object

Here, Node(data) creates a new object of the Node class, and new_node now holds a reference to this new node.

```
def print_list(self):
    current = self.head
    while current:
        print(current.data, end = " → ")
        current = current.next
    print("None").

# Create a linked list
ll = LinkedList()
ll.append(1)
ll.append(2)
ll.append(3)
ll.print_list() # output: 1 → 2 → 3 → None
```

Explainations

1. Node Class

→ The Node class represents an individual elements of the linked list. Each node contains:

- data : The actual data stored in the node (for example 1, 2, 3)
- next : A pointer (reference) to the next node in the linked list, initially, it is set to None, indicating that the node is not connected to any other node.

What does self.data = data mean?

- self : Refers to the instance of the object being created. In Python, when you define a method (like __init__), the 1st parameter is always self. It's a reference to the current object

PAGE NO.....
DATE / /

object instance, allowing you to access its attributes and methods.

- data : This is the parameter passed to the `__init__` method when creating a new `Node` object. It represents the actual value you want to store in the node. (Value Pass করে)

- self.data : This is the attribute of the `Node` object. It is a variable that belongs to the current instance of the class. In this case, `self.data` will hold the value of the `data` parameter. (value টা hold/store করে)

How it Works?

→ When you create a new instance of the `Node` class

Python

```
node1 = Node(10)
```

→ The `__init__` method is called with `data` set to `10`. Inside the constructor, the line `self.data = data` means:

- Set the instance variable `data` (ie. `self.data`) to the value of the `data` parameter passed to the constructor.

- For this particular case, it assigns `self.data = 10`

So, the `node1` object will have an attribute `data` with the value 10.

python
print (node1.data) # output: 10

2. Linked List Class

-> The linked list class is responsible for managing the list of nodes. It contains method

- append(data): Add a new node to the end of the list
- print_list(): Traverse and print the list from the head to the last node.

(a) append(data) Method.

-> The append method adds a new node with the specified data to the end of the linked list.

a.(1) Create a new node with the provided data

new_node = Node(data)

- This creates an object of the Node class
- ~~Since Node is defined as:~~

e.g. python
new_node = Node(5)
Now, new_node represents:
[5] -> None
(A node storing 5 with next = None)

(b) a.(2) if not self.head:
self.head = new_node(data).

- self.head is an attributes of the Linked list class, not the Node class

PAGE NO.....
DATE / /

- `self.head` stores a reference to the 1st node of the list.

- `If not self.head:` checks if the list is empty (i.e. self.head = None)

↓ ↓

→ when we check `if not self.head:`, it is the same as checking:

`if self.head is None:`

→ Are attributes of a constructor always lists?

↳ No, A constructor (--init--) in Python can store attributes of any data type, not just lists.

→ Here self.next = none, means `self.next` is initially `None`, but it will store a reference to another `Node` object.

e.g. list = [] } similar to this.
integer = 0 }

→ self.head = new_node, if the list is empty (~~list~~ not list, node holding an object reference)

↳ `self.head` is an attributes of the `LinkedList` class, not the `Node` class

↳ `self.head` stores a reference to the first node of the list.

↳ when we set `self.head = new_node`, we are assigning the reference of `new_node` (the newly created `Node` object) to `self.head`

T.B.C

PAGE NO.....
DATE / /

यो भन्दा अगाडि sort

```
def append (self, data)
```

Here Class Node is not inherited by Class linkedlist then how we can pass argument in method of Class linkedlist 'def append (self, data)'

→ In your code, the linked list class uses the Node class to create nodes, but it does not inherit from it. This called composition, where one class (in this case, linked list) uses objects of another class (Node) as part of its structure.

What is a Node Object?

→ A node is a basic building block of a linked list Each [node] object contains:

1. Data (value inside the node)
2. A reference (next) to the next node in the list (or None if it's the last node).

Example!

```
new_node = Node(10) # Creating a Node with value 10.
```

This means:

→ A new object of the [Node] class is created
→ The [__init__] method runs
→ self.data = 10
→ self.next = None (by default).

In memory, this look like:

node 1: [10 | next -> None]

यो भन्दा अगाडि End

T. B. C

PAGE NO.....
DATE / /

Key Understanding: `self.head` Stores a Refarance to `new_node`

- `new_node` is not the value (data), it's an object refarance.
- When we do `self.head = new_node`, `self.head` now points to the same object that new_node was pointing to.
- Lets say new_node = Node[5] → [5/Next → No

Then self.head = new Node

↓

[head → 5/Next → None]

- The linked list's structure now becomes

head → [5] → None

a (3)

(c) return

→ The return statement exist the method immediately after setting the head. This is crucial because if the list was empty, there's no need to traverse the list or do anything else.

a (4)

(b) Traverse to the End of the List

```
last_node = self.head
while last_node.next:
    last_node = last_node.next
```

a (4) ↓

(d.1) Initialize last_node:

↳ last_node = `self.head` sets `last_node` to point to the first node in the linked list (the head)

↳ At this point, last_node is the same as self.head.

~~d)~~ Traverse the list:

a (4).2. The `while last_node.next:` loop checks if the current node (`last_node`) has a next node.

- If `last_node.next` is not `None`, it means there is another node after the current one, so the loop continues.
- Inside the loop, `last_node = last_node.next` moves `last_node` to the next node in the list.

a.(4).3

~~f.3~~) Stop condition:

- The loop when `last_node.next` is `None`. This means last_node is now pointing to the last node in the list (the node that does not point to any other node).

a.(4).4

~~e~~ Appending Process.

`last_node.next = new_node`

what is last_node?

→ After the traversal loop, `last_node` is pointing to the last node in the linked list.

→ The last node is the node whose next pointer is None. This means it doesn't point to any other node, so it's the end of the list.

what is new_node?

→ new_node is the node you want to add to the end of the list. It is created using:

new_node = Node(data)

PAGE NO.....
DATE / /

what does `last_node.next = new_node` do?

→ last_node.next is the next pointer of the last node in the list.

→ Initially, `last_node.next` is `None` because the last node doesn't point to anything.

what happens after this line?

↳ The `new_node` is now connected to the end of the list

↳ The `next` pointer of the previously last node (which is None) now points to the `new_node`

↳ The `new_node` becomes the new last node in the list because its next pointer is still None.

(b) Print list

Purpose of print_list

→ The print_list method is used to traverse the linked list and print the data stored in each node. It start from the head of the list and moves through each node until it reaches the end (None), printing the data along the way.

PAGE NO.....
DATE / /

b. (1) Initialize current (current = self.head)

→ `current` is a temporary variable that starts at the head of the linked list.

→ `self.head` points to the first node in the list.

b. (2) Traverse the list with a `while` loop:
(while current:)

→ The loop continues as long as `current` is not `None`.

→ When `current` becomes `None`, it means we've reached the end of the list, and the loop stops.

b (3) Print the data of the current node:
print (current.data, end = " -> ")

→ `current.data` accesses the data stored in the current node.

→ `end = " ->"` ensures that the output is printed on the same line, with an arrow (->) separating the data of each node.

b. (4) Move to the next node:
current = current.next

→ `current.next` points to the next node in the list.

-> By setting `current = current.next`, we move the `current` pointer to the next node.

b (5) Print "None" at the ends

```
print("None")
```

-> After the loop ends (when current becomes None), we print "None" to indicate the end of the Linked List.

(3)

```
ll = LinkedList()
ll.append(1)
ll.append(2)
ll.append(3)
ll.print_list()

# output: 1 -> 2 -> 3 -> None
```

Initial Setup

we start with an empty linked list:

```
ll = LinkedList()
```

At this point,

- ll.head is None (the list is empty).

First append(1) Call

```
ll.append(1)
```

what happens?

(1) • new_node = Node(1) creates a node with data 1 and next set to None.

② Check if the list is empty

- `if not self.head` : evaluates to True because self.head is None.
- Set `self.head = new_node` . Now, `self.head` points to the node with data 1

③ Return

- The method exits immediately because the list was empty.

State of the list after append(1):

Head -> 1 -> None

Second append(2) call.

ll.append(2)

What happens?

① create a new node:

- new_node = Node(2) creates a node with data 2 and `next` to set to `None`

② Check if the list is empty:

- if `not self.head:` evaluates to False because self.head points to the node with data 1.
- skip the if block.

PAGE NO.....
DATE / /

3. Traverse to the end of the list

→ start at self.head (node with data 1).

→ `last_node = self.head` sets `last_node` to point to the node with data 1.

→ `while last_node.next`: check if `last_node.next` is None. It is None. so the loop doesn't run.

Status of the list after `append(2)`:

Head -> 1 -> 2 -> None

similarly --- for Third append.(3) self

Head -> 1 -> 2 -> 3 -> None

Relation of def LinkedList and print_list